

Development, programming and test of a modular 2.4 GHz LoRa RF link

Auteur : Loeffel Eliott

Superviseurs : Marc-Olivier Arrigo | Maribel Caceres Rivera

Professeur : Yves Perriard

3 Juin 2026

Table des matières

1	Modulation LoRa et Architecture Hardware	5
1.1	Contexte du projet	5
1.2	Objectif du projet	5
1.3	Qu'est-ce que LoRa ?	6
1.3.1	Paramètres de configuration LoRa	6
1.3.2	Sensibilité du récepteur SX1280	7
1.4	Architecture matérielle et logicielle	8
1.4.1	Vue d'ensemble du système	8
1.4.2	Composants du système	9
2	Implémentation	10
2.1	Interface en ligne de commande (CLI)	10
2.1.1	Commande <code>send()</code>	10
2.1.2	Commande <code>get()</code>	11
2.1.3	Protocole binaire	11
2.2	Logique d'implémentation	12
2.2.1	Nœud émetteur (TX)	12
2.2.2	Nœud récepteur (RX)	13
2.2.3	Machine d'états (FSM)	15
3	Bilan de liaison	16
3.1	Équation de Friis et pertes en espace libre	16
3.2	Paramètres du bilan	17
3.2.1	Antennes utilisées	17
3.2.2	Marge de liaison	17
4	Application numérique et mesures	18
4.1	Dispositif expérimental	18
4.2	Traitement des données	20
4.3	Estimation de la marge de liaison	21
4.3.1	Calcul des pertes en espace libre	21

4.3.2	Puissance reçue théorique	21
4.3.3	Marge de liaison estimée	21
4.4	RSSI mesuré et marge calculée	21
4.5	Comparaison théorie / mesures	22
5	Conclusion	23
5.1	Bilan du projet	23
5.2	Difficultés rencontrées	24
5.2.1	Mise en service des modules radio	24
5.2.2	Portage vers l'environnement STM32	24
5.3	Perspectives et axes d'amélioration	25
A	Code – Initialisation module SX1280	26
B	Code source – Nœud TX	27
C	Code source – Nœud RX	29
D	Mesures RSSI – Balayage complet des paramètres LoRa	31

Table des figures

1.1	Spectrogramme temps-fréquence de la modulation LoRa (CSS) – Trois configurations sont comparées : SF7/BW125 kHz, SF9/BW250 kHz, SF11/BW500 kHz – Source : All about LoRa and LoRaWAN, https://www.sghosly.com/p/lora_6.html . .	7
1.2	Architecture globale du système de communication LoRa.	8
2.1	Machine d'états complète de la logique de communication. À gauche : FSM du nœud TX. À droite : FSM du nœud RX.	15
4.1	Dispositif de mesure sur le toit. Les deux mâts supportent les antennes dipôles, séparés de 3 m et disposés à 1.5 m de hauteur.	19
4.2	Gros plan sur le montage antenne. Le module est fixé sur un support, avec l'antenne dipôle orientée verticalement.	20

Liste des tableaux

1.1	Sensibilité du récepteur SX1280 en mode LoRa (dBm)	8
1.2	Récapitulatif des protocoles utilisés dans le système	9
1.3	Caractéristiques clés du module Ebyte E28-2G4M12XX	9
3.1	Paramètres du bilan de liaison	17
4.1	Récapitulatif du bilan de liaison – comparaison théorique et mesurée	22
D.1	Récapitulatif des mesures RSSI pour les 128 configurations LoRa (50 paquets par configuration, distance fixe de 3 m en espace libre)	31

Chapitre 1

Modulation LoRa et Architecture Hardware

1.1 Contexte du projet

Le Center for Artificial Muscles (CAM) de l'EPFL développe un dispositif d'assistance cardiaque innovant consistant en un muscle artificiel souple enroulé autour de l'aorte, capable de se contracter et de se dilater pour augmenter la fonction naturelle du vaisseau et aider le cœur à propulser le sang. Contrairement aux technologies cardiaques actuelles qui nécessitent une connexion directe au cœur via une chirurgie invasive, ce dispositif n'interagit pas directement avec le cœur mais agit sur l'artère. L'actionneur utilisé est un actionneur élastomère diélectrique (DEA) – un polymère qui convertit l'énergie électrique en travail mécanique – placé autour de l'aorte près de la valve aortique. Dans ce cadre, il est nécessaire de pouvoir monitorer en temps réel les grandeurs électriques appliquées à l'actionneur depuis l'extérieur du corps, afin de garantir un fonctionnement correct du dispositif implanté.

1.2 Objectif du projet

Dans le cadre du projet d'assistance cardiaque, il est nécessaire d'établir une transmission bidirectionnelle sans fil des données, telles que le courant et la tension appliqués en temps réel à l'actionneur. Un terminal externe doit pouvoir surveiller ces signaux afin de garantir le bon fonctionnement de l'actionneur. Le protocole LoRa 2.4 GHz a été retenu pour sa faible consommation et sa robustesse au bruit, le rendant adapté à la communication avec un module cardiaque implanté.

Ce rapport présente l'étude et la validation d'un système de communication sans fil utilisant la technologie LoRa (*Long Range*) à 2.4 GHz, implémenté sur deux modules Ebyte E28-2G4M12XX intégrant la puce Semtech SX1280. L'objectif est de concevoir, programmer et caractériser expérimentalement une liaison radio bidirectionnelle avec mécanisme d'acquittement (ACK).

Le système complet comprend :

- Un PC terminal servant d'interface utilisateur
- Deux modules ESP32-WROOM-32 comme intermédiaires UART/SPI
- Deux modules SX1280 pilotés via SPI
- Une liaison radio LoRa à 2.4 GHz entre les deux SX1280

1.3 Qu'est-ce que LoRa ?

LoRa (*Long Range*) est une technique de modulation radio à étalement de spectre développée par Semtech. Elle repose sur la modulation CSS (*Chirp Spread Spectrum*) : le signal émis est un chirp, c'est-à-dire un signal dont la fréquence varie linéairement dans le temps. Cette approche confère à LoRa plusieurs propriétés remarquables :

- **Robustesse au bruit** : le signal étalé peut être décodé même en dessous du plancher de bruit
- **Immunité aux interférences** : la corrélation avec le chirp de référence rejette les signaux non-LoRa
- **Longue portée** : selon les paramètres, plusieurs kilomètres en espace libre
- **Faible consommation** : adapté aux objets connectés sur batterie

1.3.1 Paramètres de configuration LoRa

Le comportement de la modulation LoRa est entièrement défini par trois paramètres fondamentaux.

Facteur d'étalement (SF — *Spreading Factor*)

Le facteur d'étalement SF (SF5 à SF12 pour le SX1280) détermine le nombre de chips par symbole :

$$N_{\text{chips/symbole}} = 2^{\text{SF}} \quad (1.1)$$

Un SF élevé augmente la durée du symbole, ce qui améliore la sensibilité du récepteur mais réduit le débit utile. Un SF faible favorise le débit au détriment de la portée.

Largeur de bande (BW — *Bandwidth*)

La largeur de bande est équivalente au débit en chips (*chip rate*). Le SX1280 supporte : 203 kHz, 406 kHz, 812 kHz et 1625 kHz. La durée d'un symbole est donnée par :

$$T_s = \frac{2^{\text{SF}}}{\text{BW}} \quad (1.2)$$

Une BW large augmente le débit et réduit le temps d'émission, mais dégrade la sensibilité. La figure 1.1 illustre le spectrogramme temps-fréquence de la modulation LoRa pour différentes combinaisons SF/BW.

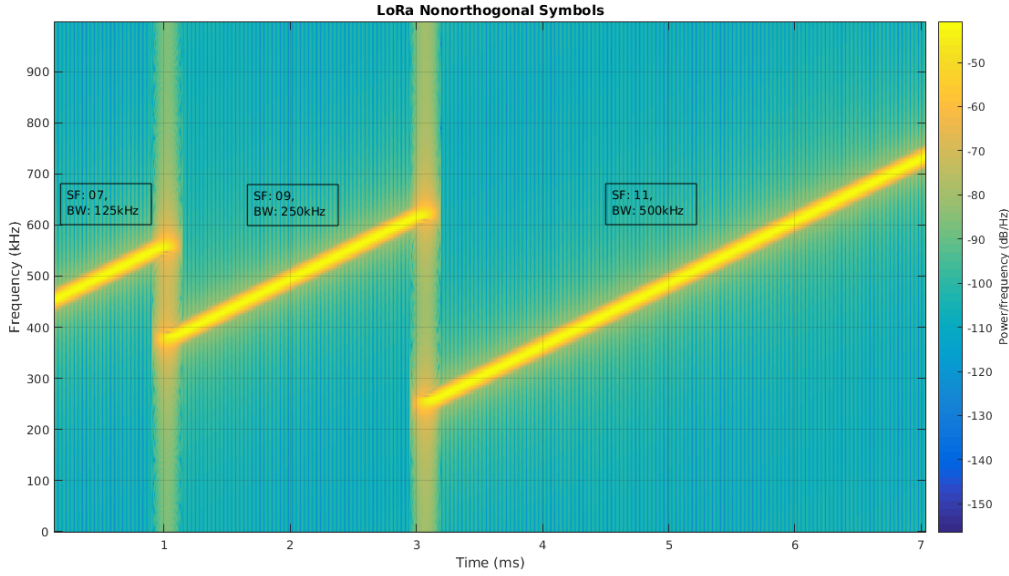


Figure 1.1 – Spectrogramme temps-fréquence de la modulation LoRa (CSS) – Trois configurations sont comparées : SF7/BW125 kHz, SF9/BW250 kHz, SF11/BW500 kHz – Source : All about LoRa and LoRaWAN, https://www.sghosly.com/p/lora_6.html

Taux de codage (CR — *Coding Rate*)

Le taux de codage CR détermine le niveau de redondance FEC (*Forward Error Correction*). Les valeurs disponibles sont 4/5, 4/6, 4/7 et 4/8. Un CR élevé améliore la correction d'erreurs mais réduit le débit effectif :

$$R_b = \frac{SF \cdot BW}{2^{SF}} \cdot \frac{4}{CR} \quad (1.3)$$

Tous ces paramètres doivent être identiques entre les modules LoRa qui souhaitent communiquer. Le choix de ces derniers résulte d'un compromis entre sensibilité et débit de données transmis, et nécessite un dimensionnement adapté à l'application visée.

1.3.2 Sensibilité du récepteur SX1280

Le tableau 1.1 présente les sensibilités du récepteur SX1280 en mode LoRa (PER = 1%, 10 octets de payload, 25°C, CR = 4/5, d'après la datasheet Semtech).

Table 1.1 – Sensibilité du récepteur SX1280 en mode LoRa (dBm)

SF	BW = 203 kHz	BW = 406 kHz	BW = 812 kHz	BW = 1625 kHz
SF5	−109	−107	−105	−99
SF6	−111	−110	−108	−103
SF7	−115	−113	−112	−106
SF8	−118	−116	−115	−109
SF9	−121	−119	−117	−111
SF10	−124	−122	−120	−114
SF11	−127	−125	−123	−117
SF12	−130	−128	−126	−120

Source :

SX1280 Datasheet Rev 1.1, Semtech, Tableau 6-1.

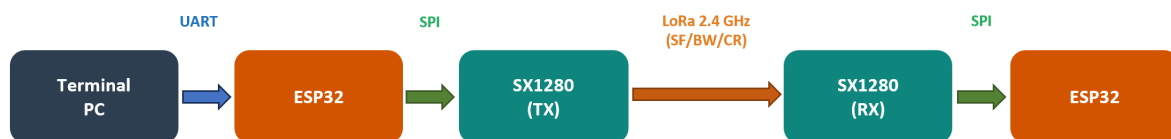
Les configurations extrêmes utilisées dans ce projet sont les suivantes :

- **Configuration haute sensibilité** : SF12, BW=203 kHz → sensibilité −130 dBm (mode LP) ou −132 dBm (mode HS)
- **Configuration haut débit** : SF7, BW=1625 kHz → sensibilité −106 dBm (mode LP) ou −109 dBm (mode HS)

1.4 Architecture matérielle et logicielle

1.4.1 Vue d'ensemble du système

L'architecture du système est représentée à la figure 1.2. Elle met en œuvre cinq composants connectés via trois protocoles standards.

**Figure 1.2** – Architecture globale du système de communication LoRa.

Le système est bidirectionnel et permet à l'émetteur d'envoyer des commandes à l'actionneur ou de recevoir des données des capteurs. Une interface en ligne de commande (CLI) a été développée pour simplifier le dialogue entre le PC et le nœud distant, ainsi que pour faciliter le debug lors des phases de conception.

Table 1.2 – Récapitulatif des protocoles utilisés dans le système

Liaison	Protocole	Débit	Rôle
PC ↔ ESP32	UART	115 200 baud	Commandes utilisateur
ESP32 ↔ SX1280 TX	SPI	Jusqu'à 18 MHz	Pilotage module radio
SX1280 TX ↔ SX1280 RX	LoRa 2.4 GHz	0.476–202 kb/s	Transmission radio
ESP32 ↔ SX1280 RX	SPI	Jusqu'à 18 MHz	Lecture données reçues

1.4.2 Composants du système

Module radio Ebyte E28-2G4M12XX (SX1280)

Le module Ebyte E28-2G4M12XX est un module radio compact intégrant la puce Semtech SX1280/SX1281. Ses caractéristiques principales sont récapitulées dans le tableau 1.3.

Table 1.3 – Caractéristiques clés du module Ebyte E28-2G4M12XX

Paramètre	Valeur
Fréquence centrale	2.4 GHz (2400–2500 MHz)
Puissance TX maximale	+12 dBm (programmable)
Sensibilité RX (SF12)	−130 dBm @ BW = 203 kHz
Interface	SPI (mode 0, jusqu'à 18 MHz)
Tension d'alimentation	1.8–3.7 V
Courant TX (typ.)	100 mA @ +10 dBm
Courant RX (typ.)	5.5–8.2 mA selon BW
Courant sleep	0.2 µA
Modulations supportées	LoRa, FLRC, FSK (GFSK)
Antenne	Connecteur µFL / pad PCB

Module ESP32-WROOM-32

L'ESP32-WROOM-32 assure la passerelle entre le PC (liaison série UART) et le module SX1280 (SPI). Il reçoit les commandes depuis le terminal PC, les formate et les transmet au SX1280 via SPI à l'aide de la librairie RadioLib.

Librairie RadioLib

RadioLib est une librairie open-source (jgromes/RadioLib sur GitHub) qui fournit une abstraction haut niveau pour le pilotage de nombreux modules radio, dont le SX1280. Elle gère la configuration des registres, les modes de fonctionnement (TX, RX, sleep) et la gestion des interruptions via les signaux DIO.

Chapitre 2

Implémentation

2.1 Interface en ligne de commande (CLI)

L'interface en ligne de commande constitue le point d'entrée logiciel de la chaîne de communication LoRa. Elle permet à un opérateur PC d'envoyer des commandes ou d'interroger des variables sur le nœud distant, sans avoir à manipuler directement le protocole série binaire. L'outil est écrit en Python et s'appuie sur la bibliothèque Typer pour la gestion des arguments, ainsi que sur PySerial pour la communication UART. Le projet se compose de deux fichiers :

1. **main.py** – point d'entrée de la CLI. Il définit l'application Typer et expose deux sous-commandes : *send()* et *get()*.
2. **serial_comm.py** – couche de transport. Il contient la logique d'encodage des commandes, d'écriture sur le port série et de lecture de la réponse.

2.1.1 Commande **send()**

Envoi d'une commande avec une valeur :

```
1 $ python main.py send <commande> <valeur> [--port COM5] [--baudrate  
9600]
```

Listing 2.1 – Exemple de commande send

Cette sous-commande permet d'envoyer une consigne au nœud distant. L'utilisateur fournit un nom de commande textuel (**voltage**, **current**, **temperature**, **status**) et une valeur numérique flottante. Avant transmission, la fonction `encode_command()` convertit ces deux informations en une trame binaire de 2 octets :

- Octet 1 (**cmd_byte**) : identifiant numérique de la commande, issu du dictionnaire **COMMAND** (**voltage** = 1, **current** = 2, **temperature** = 3, **status** = 4).
- Octet 2 (**value_byte**) : valeur encodée par la fonction `float2int()`, soit `int(round(valeur × 10))`.
Ce schéma permet de transmettre une décimale sans virgule flottante – par exemple, 5.0 V devient l'octet 50.

La valeur encodée est contrainte à l'intervalle $[0, 254]$, car la valeur `0xFF` (255) est réservée comme marqueur de requête GET et ne peut pas être utilisée comme valeur d'envoi.

2.1.2 Commande `get()`

Interrogation d'une variable distante :

```
1 $ python main.py get <varname> [--port COM5] [--baudrate 9600]
```

Listing 2.2 – Exemple de commande get

Cette sous-commande permet de lire la valeur courante d'une variable stockée sur le nœud distant. Elle envoie également une trame de 2 octets, mais avec une sémantique différente :

- Octet 1 : identifiant numérique de la variable demandée (même dictionnaire `COMMAND`).
- Octet 2 : la constante `0xFF` (`GET_FLAG`), qui signale au firmware que c'est une requête de lecture et non une commande d'écriture.

Après émission, la fonction `getData()` ouvre une fenêtre d'attente de 2 secondes durant laquelle elle scrute le port série ligne par ligne. Elle attend une réponse de la forme `GET:<cmd>:<valeur>`, décode la valeur brute (division par 10), puis l'affiche.

2.1.3 Protocole binaire

La CLI produit systématiquement des trames de 2 octets exactement, conformément à ce qu'attend le firmware côté ESP32. Ce dernier ne lit la liaison série que lorsqu'au moins 2 octets sont disponibles (`Serial.available() >= 2`), ce qui garantit que les deux octets arrivent ensemble et sont interprétés de façon cohérente. La distinction entre une commande *send* et une requête *get* repose uniquement sur la valeur du second octet : toute valeur de 0 à 254 est interprétée comme une écriture, tandis que `0xFF` déclenche une lecture. Les options `-port` et `-baudrate` ont pour valeurs par défaut `COM5` et `9600`, mais peuvent être surchargées à l'appel pour s'adapter aux différentes configurations matérielles.

2.2 Logique d'implémentation

Le but de cette section est de présenter dans sa globalité la logique implémentée pour la transmission de messages. L'intégralité du code se trouve sur le repository GitHub¹ du projet. Seuls les points importants de chaque partie sont mentionnés dans ce rapport ; des versions simplifiées de la logique pour chaque nœud sont disponibles en annexe.

2.2.1 Nœud émetteur (TX)

Le programme côté émetteur constitue le firmware de l'ESP32 côté PC. Son rôle est d'être l'intermédiaire entre le terminal Python (la CLI) et le réseau LoRa : il reçoit les commandes arrivant par liaison série UART, les retransmet par radio au nœud distant, attend un accusé de réception, puis renvoie le résultat à la CLI. Il est développé en C++ sous PlatformIO avec la bibliothèque RadioLib 7.6 et cible un module radio SX1280.

Initialisation TX

Au démarrage (`setup()`), le programme initialise deux interfaces :

1. La liaison série UART
2. Le module radio SX1280, instancié sur les broches SPI dédiées :

```
1 SX1280 radio = new Module(25, 14, 2, 13); // NSS=25, DIO1=14,
    NRST=2, BUSY=13
```

Un CRC sur 2 octets est activé pour garantir l'intégrité des paquets reçus. La broche DIO1 est configurée en interruption hardware via `radio.setDio1Action(setFlag)` : à chaque fin d'opération radio (émission terminée ou paquet reçu), le flag volatile `operationDone` est levé par la routine d'interruption `setFlag()`, signalant à la boucle principale qu'une action peut être traitée.

Boucle principale TX

TX_NODE :

Le firmware déclenche l'émission LoRa du tableau `dataArr` (2 octets) via `radio.startTransmit()`. L'émission est non bloquante : elle retourne immédiatement, et c'est l'interruption DIO1 qui signalera sa fin. Si un nouvel octet arrive sur le port série pendant cet état, il est ignoré – aucune nouvelle commande ne peut interrompre une transmission en cours. L'état passe immédiatement à *RX_NODE*.

1. https://github.com/DolbyBolby/LoRa_CAM

WAIT_ACK :

C'est l'état le plus critique. Le firmware attend la réponse du nœud distant dans une fenêtre de 200 ms. Deux chemins sont possibles :

1. Timeout dépassé : aucun paquet reçu dans le délai imparti. Le firmware affiche "TIMEOUT, no message received, retrying..." et retourne à *TX_NODE* pour retransmettre automatiquement la commande.
2. Paquet reçu (DIO1 levé) : le paquet est lu avec `radio.readData()`. La logique de validation dépend du type de requête :
 - . Requête SEND (`dataArr[1] != 0xFF`) : les deux octets de l'écho doivent correspondre exactement à ceux envoyés.
 - . Requête GET (`dataArr[1] == 0xFF`) : seul le premier octet (identifiant de commande) est comparé ; le second octet de la réponse contient la valeur stockée sur le nœud distant.

Dans les deux cas, un octet de confirmation est renvoyé par radio au nœud distant : `0xAA` si la validation est réussie, `0xFF` en cas de désaccord. L'état passe à *SEND_CONFIRM*.

Robustesse et mécanisme de retry

Le firmware implémente trois niveaux de résilience :

1. **Timeout de 200 ms** : si le nœud distant ne répond pas, la commande est retransmise automatiquement, sans intervention de l'opérateur.
2. **CRC sur 2 octets** : les paquets corrompus sont silencieusement ignorés en vérifiant que le code de retour de la radio est égal à `RADIOLIB_ERR_NONE`.
3. **Validation des données reçues** : si les octets reçus ne correspondent pas à ceux envoyés (corruption radio malgré le CRC), la commande est également retransmise.

Ces trois mécanismes forment une boucle de confirmation fiable qui garantit, dans des conditions radio normales, que chaque commande est bien prise en compte par le nœud distant avant que le firmware ne retourne en attente.

2.2.2 Nœud récepteur (RX)

Le programme côté récepteur est le firmware de l'ESP32 côté nœud distant. Son rôle est symétrique à celui de l'émetteur : il écoute en permanence le canal radio LoRa, traite les paquets entrants selon leur type (commande d'écriture ou requête de lecture), maintient un magasin de variables internes, renvoie une réponse à l'émetteur, puis attend la confirmation de bonne réception avant de revenir en écoute. Malgré son appellation, ce firmware émet également des données radio – il serait plus juste de le qualifier de nœud esclave bidirectionnel.

Initialisation RX

L'initialisation de la radio est identique à celle du nœud émetteur et doit utiliser exactement les mêmes paramètres : toute divergence de configuration rendrait la liaison radio silencieuse. Un CRC sur 2 octets est activé et la broche DIO1 est configurée en interruption via `radio.setDio1Action(setFlag)`. À la différence de l'émetteur, le récepteur arme la réception dès la fin du `setup()` avec `radio.startReceive()`, ce qui signifie qu'il est immédiatement prêt à recevoir un paquet sans attendre un événement externe.

Variables internes

Une caractéristique propre à ce nœud est la présence d'un tableau de variables persistantes en mémoire RAM :

```

1 static SensorVar store[] = {
2     {1, 0},    // voltage    (0.0 V)
3     {2, 0},    // current    (0.0 A)
4     {3, 250},  // temperature (25.0 C)
5     {4, 10},   // status     (1.0)
6 };

```

Chaque entrée associe un opcode de commande à une valeur encodée au format $\times 10$ (un entier représentant une décimale, ex. 250 $\rightarrow$ 25.0). Ce magasin remplit deux fonctions :

1. Il mémorise les valeurs reçues lors d'une commande SEND, pour pouvoir les restituer lors d'une requête GET ultérieure.
2. Il fournit des valeurs initiales par défaut au démarrage du nœud (notamment `temperature = 25.0`).

Deux fonctions accesseurs opèrent sur ce tableau : `getStoredValue(opcode)` pour lire une entrée, et `updateStoredValue(opcode, value)` pour la mettre à jour.

Boucle principale RX

WAIT_CONFIRM :

Le nœud distant attend la réponse de l'émetteur – un unique octet de confirmation :

1. **0xAA** : la transaction a été validée. Le nœud affiche sur son port série local la commande et la valeur traitées, remet `dataArr` à zéro et retourne à *WAIT_MSG*.
2. **0xFF ou lecture invalide** : la transaction a été rejetée. Le nœud affiche "[REJECTED by TX]" et retourne à *WAIT_MSG* sans modifier le magasin.
3. **Timeout de 300 ms** : si aucune confirmation n'arrive dans le délai imparti, le nœud affiche "[TIMEOUT waiting for confirm]" et retourne à *WAIT_MSG*. Ce timeout est intentionnellement plus long que le timeout de 200 ms de l'émetteur, pour lui laisser le temps d'envoyer sa confirmation après avoir reçu l'écho.

Dans tous les cas, `radio.startReceive()` est réarmé avant de retourner à `WAIT_MSG`, garantissant que le nœud ne rate pas le paquet suivant.

2.2.3 Machine d'états (FSM)

Le protocole applicatif implémente un mécanisme de communication fiabilisé avec acquittement (ACK), timeout et retransmission. Il est structuré autour d'une machine d'états finis (FSM) exécutée en parallèle sur les deux nœuds.

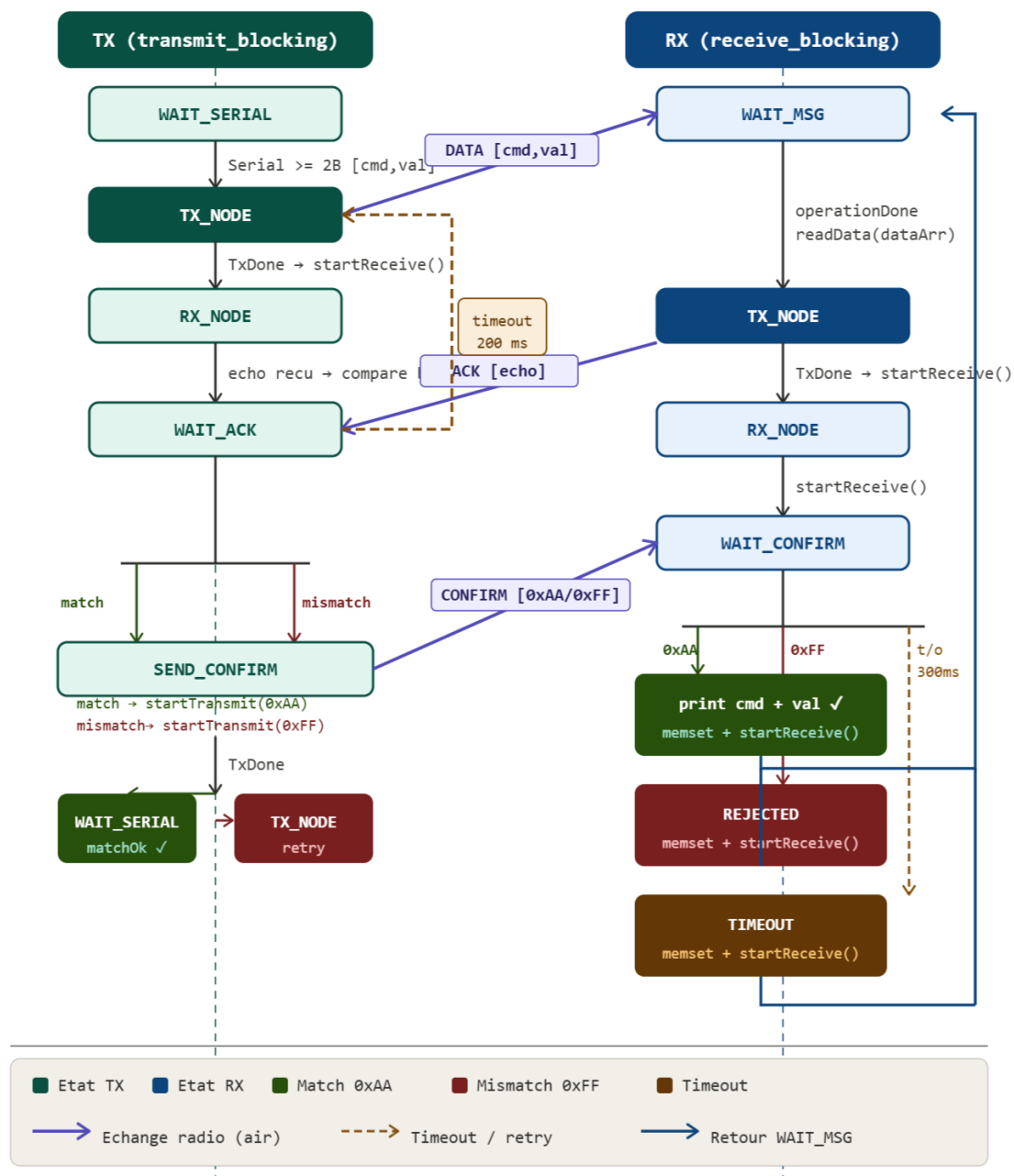


Figure 2.1 – Machine d'états complète de la logique de communication. À gauche : FSM du nœud TX. À droite : FSM du nœud RX.

Chapitre 3

Bilan de liaison

La particularité fondamentale de ce scénario de communication réside dans la nature du milieu de propagation. Le module émetteur est fixé sur la paroi de l’aorte, à l’intérieur de la cage thoracique. Le signal doit donc traverser plusieurs centimètres de tissus biologiques – peau, graisse, muscle – avant d’atteindre le récepteur externe positionné à environ 2 mètres du patient. Les tissus biologiques sont des milieux fortement atténuants à 2.4 GHz : leur permittivité élevée et leurs pertes diélectriques significatives engendrent une atténuation supplémentaire importante par rapport à une propagation en espace libre, rendant ce scénario fondamentalement différent d’une liaison LoRa conventionnelle.

Cette situation étant difficile à reproduire expérimentalement sans protocole dédié, le bilan de liaison constitue l’outil privilégié pour répondre à la question centrale du projet : le lien LoRa 2.4 GHz est-il viable à travers les tissus du corps humain ? Il s’agit de quantifier, étape par étape, toutes les contributions au bilan de puissance – puissance émise, gains d’antenne, pertes en espace libre, atténuation due aux tissus, marges de shadowing – pour estimer la puissance reçue et la comparer à la sensibilité du récepteur SX1280. Si la marge de liaison reste positive, même dans le pire cas, la communication est théoriquement faisable. Cette analyse est d’autant plus critique que le système est destiné à un usage médical implanté, où il n’est pas possible d’augmenter arbitrairement la puissance d’émission (réglementation, échauffement des tissus, autonomie énergétique).

3.1 Équation de Friis et pertes en espace libre

Le bilan de liaison (*link budget*) permet d’évaluer la viabilité d’une liaison radio. Il repose sur l’équation de Friis qui exprime la puissance reçue P_r en fonction des paramètres de la liaison :

$$P_r [\text{dBm}] = P_t [\text{dBm}] + G_t [\text{dBi}] + G_r [\text{dBi}] - \text{FSPL} [\text{dB}] - L_{\text{autres}} [\text{dB}] \quad (3.1)$$

avec les pertes en espace libre FSPL (*Free Space Path Loss*) définies par :

$$\text{FSPL [dB]} = 20 \log_{10}(d) + 20 \log_{10}(f) + 20 \log_{10}\left(\frac{4\pi}{c}\right) \quad (3.2)$$

La constante $20 \log_{10}(4\pi/c) \approx -147.55$ dB avec c en m/s, ce qui donne la forme numérique courante (avec d en km et f en GHz) :

$$\text{FSPL [dB]} = 20 \log_{10}(d_{\text{km}}) + 20 \log_{10}(f_{\text{GHz}}) + 93.55 \quad (3.3)$$

3.2 Paramètres du bilan

Table 3.1 – Paramètres du bilan de liaison

Symbole	Paramètre	Valeur	Source
P_t	Puissance TX	+10 dBm	SX1280 (programmable)
G_t	Gain antenne TX (dipôle)	+2.1 dBi	Datasheet antenne
G_r	Gain antenne RX (dipôle)	+2.1 dBi	Datasheet antenne
f	Fréquence	2.4 GHz	Paramètre SX1280
d	Distance de mesure	3 m	Mesure sur toit
L_{corps}	Atténuation corps humain	+30 dB	Valeur conservatrice (pire cas)
L_{multi}	Marge multi-trajets	+6 dB	Marge opérationnelle
S_{min}	Sensibilité RX (SF12/BW203)	−130 dBm	Datasheet SX1280

3.2.1 Antennes utilisées

Les mesures ont été effectuées avec des dipôles bipolaires offrant un gain de 2.1 dBi à 2.4 GHz. La littérature (FCC OET Bulletin 65 / IEEE 802.15.6) indique une atténuation due au corps humain de 22 à 30 dB à 2.4 GHz ; la valeur de 30 dB a été retenue comme cas pessimiste.

3.2.2 Marge de liaison

La marge de liaison M est définie par :

$$M [\text{dB}] = P_r - S_{\text{min}} > 0 \quad (3.4)$$

où S_{min} est la sensibilité minimale du récepteur. Une marge positive garantit la viabilité théorique du lien.

Chapitre 4

Application numérique et mesures

Les mesures expérimentales présentées dans ce chapitre ont pour objectif de valider le modèle théorique établi au chapitre précédent. Un RSSI mesuré en espace libre concordant avec la valeur estimée par l'équation de Friis ancre le modèle sur des données réelles et justifie son extrapolation au scénario d'implantation intra-corporel.

4.1 Dispositif expérimental

Les mesures ont été réalisées sur le toit d'un bâtiment, offrant des conditions proches de l'espace libre et minimisant les réflexions et les interférences. L'objectif était de balayer l'ensemble des configurations possibles des paramètres de modulation LoRa (SF/BW/CR) afin d'analyser la puissance reçue pour chacune. Pour chaque configuration, 50 mesures du RSSI (*Received Signal Strength Indicator*) ont été collectées et stockées dans un fichier CSV¹.

Le RSSI ne dépendant pas des paramètres de modulation SF, BW et CR – qui agissent sur la sensibilité et la robustesse du lien mais non sur la puissance reçue – les valeurs mesurées sont attendues constantes d'une configuration à l'autre. La confirmation de ce comportement valide que chaque configuration a correctement établi la liaison et reçu les paquets dans des conditions comparables.

1. `mesure-RSSI.csv` disponible sur le repository GitHub.

La figure 4.1 montre la configuration de mesure.



Figure 4.1 – Dispositif de mesure sur le toit. Les deux mâts supportent les antennes dipôles, séparés de 3 m et disposés à 1.5 m de hauteur.

La configuration de mesure est la suivante :

- **Distance TX–RX** : $d = 3$ m (hauteur des mâts : 1.5 m)
- **Antennes** : dipôles bipolaires, gain 2.1 dBi, fréquence 2.4 GHz
- **Métrique relevée** : RSSI (*Received Signal Strength Indicator*)

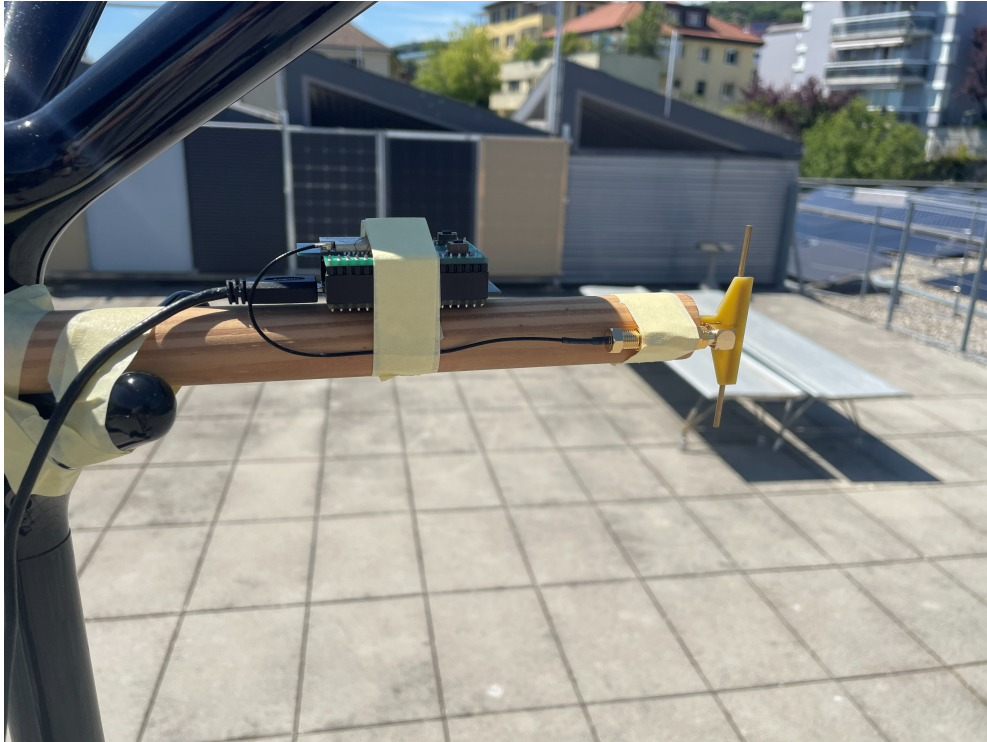


Figure 4.2 – Gros plan sur le montage antenne. Le module est fixé sur un support, avec l’antenne dipôle orientée verticalement.

4.2 Traitement des données

Pour chacune des 128 configurations, 50 mesures de RSSI ont été collectées et consignées dans un fichier dédié. Le choix de 50 mesures par configuration résulte d’un compromis entre la durée totale de l’expérience – qui implique 6 400 paquets transmis au total – et la robustesse statistique souhaitée. La valeur moyenne de ces 50 mesures est retenue comme valeur représentative de chaque configuration, ce qui permet d’atténuer l’influence des fluctuations ponctuelles du canal dues au multipath ou aux interférences ambiantes.

En complément de la moyenne, l’écart-type est calculé pour chaque configuration. Un écart-type faible confirme que le canal est stable sur la durée de la mesure et que la moyenne est statistiquement représentative. Le tableau résumant les RSSI moyens et les écarts-types pour chaque configuration se trouve en annexe.

Le RSSI moyen global s’établit à -46.21 dBm, avec une plage totale de variation de seulement 4.98 dB entre la valeur la plus favorable (-44.04 dBm, config. 69 : SF9/BW406/CR4/5) et la plus faible (-49.02 dBm, config. 64 : SF8/BW1625/CR4/8). L’écart-type moyen sur l’ensemble des 128 configurations est de 0.56 dB, valeur très faible qui traduit une excellente stabilité de la réception paquet à paquet.

4.3 Estimation de la marge de liaison

4.3.1 Calcul des pertes en espace libre

Application numérique de l'équation (3.3) pour $d = 3 \text{ m} = 0.003 \text{ km}$, $f = 2.4 \text{ GHz}$:

$$\text{FSPL} = 20 \log_{10}(0.003) + 20 \log_{10}(2.4) + 93.55 = 50.68 \text{ dB} \quad (4.1)$$

4.3.2 Puissance reçue théorique

En espace libre (sans atténuation supplémentaire) :

$$P_r = 10 + 2.1 + 2.1 - 50.68 = -36.48 \text{ dBm} \quad (4.2)$$

4.3.3 Marge de liaison estimée

En intégrant les pertes additionnelles ($L_{\text{corps}} = 30 \text{ dB}$, $L_{\text{multi}} = 6 \text{ dB}$) :

$$M_{\text{est}} = -36.48 - 36 + 130 = 57.52 \text{ dB} \quad (4.3)$$

Interprétation : une marge de 57 dB est très confortable, indiquant que le lien reste viable même dans les conditions les plus dégradées. La configuration choisie optimise la sensibilité des émetteurs-récepteurs. Cette marge offre également la possibilité d'augmenter la bande passante et de réduire le facteur d'étalement, ce qui réduirait la sensibilité mais augmenterait le débit de données si la situation le nécessitait.

4.4 RSSI mesuré et marge calculée

Pour ce calcul, on utilise la valeur centrale de l'ensemble des moyennes RSSI mesurées lors de l'expérience :

$$\text{RSSI}_{\text{meas}} = -46.21 \text{ dBm} \quad (4.4)$$

La marge de liaison calculée à partir du RSSI mesuré est :

$$M_{\text{cmp}} = -\text{RSSI}_{\text{meas}} - 30 + 130 = 46.21 - 30 + 130 = 146.21 \text{ dB} \quad (4.5)$$

Les 6 dB de pertes dues aux connecteurs ne sont pas ajoutés séparément car ils sont déjà inclus dans la puissance de signal mesurée.

4.5 Comparaison théorie / mesures

Table 4.1 – Récapitulatif du bilan de liaison – comparaison théorique et mesurée

Grandeur	Valeur théorique	Valeur mesurée
FSPL @ $d = 3\text{ m}$, $f = 2.4\text{ GHz}$	50.68 dB	—
RSSI	−36.48 dBm	−46.21 dBm
Marge de liaison	57.52 dB	53.79 dB
Sensibilité SX1280 (SF12/BW203)	−130 dBm	−130 dBm

Analyse des résultats

L'écart de $\sim 10\text{ dB}$ entre la puissance reçue théorique (−36.48 dBm) et le RSSI mesuré (−46.21 dBm) est cohérent avec les pertes réelles en environnement semi-réverbérant (toit avec surfaces réfléchissantes, multi-trajets). La marge de liaison mesurée confirme une liaison largement viable dans les conditions de test.

Cette marge offre une flexibilité dans le choix des paramètres SF/BW/CR, permettant de sacrifier une partie de la sensibilité au profit d'un meilleur débit si les besoins applicatifs le nécessitent.

Chapitre 5

Conclusion

5.1 Bilan du projet

Ce projet avait pour objectif de concevoir, programmer et valider expérimentalement un système de communication sans fil LoRa 2.4 GHz dans le cadre du projet d’assistance cardiaque du Center for Artificial Muscles (CAM) de l’EPFL. L’enjeu central était d’évaluer la faisabilité d’une liaison radio entre un module implanté au niveau de l’aorte et un terminal externe, à travers les tissus du corps humain.

L’ensemble de la chaîne de communication a été implémentée et validée avec succès. Côté matériel, deux modules Ebyte E28-2G4M12XX intégrant la puce SX1280 ont été mis en œuvre, pilotés par des microcontrôleurs ESP32-WROOM-32 via SPI et interfacés à un terminal PC via UART. Côté logiciel, un protocole applicatif robuste a été développé en C++ sous PlatformIO, structuré autour d’une machine d’états parallèle sur les deux nœuds. Ce protocole implémente un mécanisme d’acquiescement (ACK) avec timeout de 200/300 ms et retransmission automatique, garantissant la fiabilité des échanges même en conditions dégradées. Une interface en ligne de commande (CLI) Python a également été développée, permettant à un opérateur PC d’envoyer des consignes ou d’interroger des variables distantes via deux sous-commandes intuitives – *send* et *get* – sans interaction directe avec le protocole binaire sous-jacent.

Le bilan de liaison théorique, calculé à partir de l’équation de Friis, donne une puissance reçue estimée de -36.48 dBm en espace libre à 3 mètres. Les mesures expérimentales réalisées sur le toit d’un bâtiment ont donné un RSSI moyen de -46.21 dBm sur l’ensemble des 128 configurations du balayage SF/BW/CR. L’écart d’environ 10 dB entre théorie et mesure est cohérent avec les pertes d’un environnement semi-réverbérant (réflexions, multi-trajets). La stabilité des mesures – écart-type moyen de 0.56 dB sur l’intégralité du balayage – valide la qualité du canal et la robustesse du système de mesure. En intégrant une atténuation corporelle conservatrice de 30 dB (cas pessimiste d’après IEEE 802.15.6 / FCC OET Bulletin 65), la marge de liaison mesurée confirme la viabilité du lien LoRa 2.4 GHz dans le scénario d’implantation visé, y compris dans le cas pessimiste retenu.

5.2 Difficultés rencontrées

5.2.1 Mise en service des modules radio

Dès le démarrage du projet, l'objectif était d'établir une communication unidirectionnelle simple afin de disposer rapidement d'une base fonctionnelle. La librairie RadioLib a été retenue pour le développement des firmwares ; cependant, les exemples de transmission et réception bloquante fournis dans le repository officiel de la librairie ne produisaient aucune communication entre les deux modules.

Face à ce constat, c'est en premier lieu le matériel qui a été remis en question. Un audit complet du montage a été réalisé : tests de continuité, analyse des signaux SPI à l'oscilloscope, et vérification du pont de résistance sélectionnant l'antenne active, qui a été re-soudé pour garantir la connexion de l'antenne PCB. Grâce au professeur Alexis Boegli, une vérification à l'analyseur de spectre a permis de confirmer que les modules émettaient bien dans la bande de fréquence attendue à 2.4 GHz. L'ensemble de ces vérifications s'est avéré concluant, orientant définitivement les investigations vers le logiciel. Il a finalement été nécessaire d'implémenter une logique basée sur les interruptions hardware pour obtenir une communication fonctionnelle, au prix d'un temps significatif consacré au diagnostic matériel.

Je tiens à remercier chaleureusement le professeur Boegli pour ses échanges généreux et ses précieuses connaissances en RF, partagées sans réserve tout au long de ce travail.

5.2.2 Portage vers l'environnement STM32

La seconde difficulté majeure est apparue lors de la tentative de portage de la communication vers la plateforme cible du projet, le STM32U385. Le support du SX1280 dans l'écosystème STM32 est très limité : aucun driver ni HAL dédié n'était disponible pour ce composant. Faute de temps suffisant pour en développer un depuis zéro, une tentative d'adaptation d'un driver existant pour le SX1278 – composant de la même famille mais destiné à la bande sub-GHz – a été entreprise. Malgré les efforts d'adaptation à la cartographie des registres du SX1280, il n'a pas été possible d'établir une liaison fonctionnelle dans le temps imparti. Face à cette impasse, il a été décidé de concentrer les ressources restantes sur l'étude de faisabilité de la communication et la caractérisation expérimentale du lien radio.

5.3 Perspectives et axes d'amélioration

Plusieurs axes d'amélioration et d'extension naturelle du travail se dégagent.

Validation en conditions réelles

L'étape la plus critique restant à franchir est la validation expérimentale du bilan en présence de tissus biologiques. Une première approche accessible consiste à réaliser des mesures sur un fantôme de tissu – une solution aqueuse de sucre, NaCl et agar-agar reproduisant les propriétés diélectriques du muscle ($\varepsilon_r \approx 52$, $\sigma \approx 1.7 \text{ S/m}$ à 2.4 GHz)¹ – afin de quantifier empiriquement l'atténuation corporelle et de confronter la valeur de 30 dB retenue dans le bilan aux mesures réelles.

Intégration sur STM32U385

Le projet a été développé sur ESP32 comme plateforme de prototypage. L'objectif final du CAM est d'intégrer la liaison radio dans l'architecture cible du projet cardiaque, qui repose sur un microcontrôleur STM32U3. Cette migration nécessite le portage de la couche SPI et de la logique RadioLib vers le HAL STM32, ainsi que l'intégration dans le RTOS sélectionné par le laboratoire.

Optimisation de la consommation énergétique

Dans un contexte d'implant médical, l'autonomie est une contrainte critique. Le SX1280 offre un courant de veille de $0.2 \mu\text{A}$ – plusieurs ordres de grandeur en dessous du courant de réception (5.5–8.2 mA). L'exploitation des modes sleep entre les cycles de transmission, couplée à un protocole de réveil synchronisé, permettrait de réduire drastiquement la consommation moyenne et d'envisager une alimentation sur batterie longue durée.

Extension du protocole applicatif

Le protocole binaire actuel sur 2 octets est fonctionnel mais limité à 4 variables et à des valeurs entières encodées au format $\times 10$. Une évolution vers une trame plus structurée – incluant un identifiant de nœud, un numéro de séquence et un champ de longueur variable – permettrait d'adresser un réseau de plusieurs capteurs et de transmettre des données continues à plus haute résolution, comme des signaux physiologiques en temps réel.

1. Valeurs d'après IT'IS Foundation / Gabriel 1996.

Annexe A

Code – Initialisation module SX1280

```
1 #include <RadioLib.h>
2
3 // Brochage : NSS, DIO1, NRESET, BUSY
4 SX1280 radio = new Module(NSS_PIN, DIO1_PIN, RESET_PIN, BUSY_PIN);
5
6 void setup_radio() {
7     int state = radio.begin(
8         2400.0,    // frequence centrale en MHz
9         203.125,  // BW en kHz (1625, 812, 406, 203)
10        12,        // SF (5 a 12)
11        5,         // CR (5=4/5, 6=4/6, 7=4/7, 8=4/8)
12        10,        // puissance TX en dBm (-18 a +12)
13        8          // taille du preambule
14    );
15    if (state != RADIOLIB_ERR_NONE) {
16        Serial.print("[ERROR] init radio: ");
17        Serial.println(state);
18        while (true);
19    }
20 }
```

Listing A.1 – Initialisation du module SX1280 via RadioLib

Annexe B

Code source – Nœud TX

```
1 typedef enum {
2     STATE_WAIT_SERIAL,
3     STATE_TX_NODE,
4     STATE_RX_NODE,
5     STATE_WAIT_ACK,
6     STATE_SEND_CONFIRM
7 } TxState;
8
9 bool transmit(uint8_t cmd, uint8_t val) {
10     uint8_t txBuf[2] = {cmd, val};
11     uint8_t rxBuf[2] = {0};
12     TxState state = STATE_TX_NODE;
13     uint32_t timeout_ms = 200;
14
15     while (true) {
16         switch (state) {
17             case STATE_TX_NODE:
18                 radio.startTransmit(txBuf, 2);
19                 waitForIRQ(RADIOLIB_IRQ_TX_DONE);
20                 radio.startReceive();
21                 state = STATE_WAIT_ACK;
22                 timeout_start = millis();
23                 break;
24
25             case STATE_WAIT_ACK:
26                 if (millis() - timeout_start > timeout_ms) {
27                     state = STATE_TX_NODE; // retransmission
28                     break;
29                 }
30         }
31     }
```

```
30     if (irqFired) {
31         radio.readData(rxBuf, 2);
32         bool match = (rxBuf[0] == txBuf[0]) && (rxBuf[1] ==
33             txBuf[1]);
34         state = STATE_SEND_CONFIRM;
35     }
36     break;
37
38 case STATE_SEND_CONFIRM:
39     uint8_t confirmByte = match ? 0xAA : 0xFF;
40     radio.startTransmit(&confirmByte, 1);
41     waitForIRQ(RADIOLIB_IRQ_TX_DONE);
42     if (match) return true;    // succes
43     state = STATE_TX_NODE;    // mismatch : retry
44     break;
45 }
46 }
```

Listing B.1 – Machine d'états TX (structure simplifiée)

Annexe C

Code source – Nœud RX

```
1 typedef enum {
2     STATE_WAIT_MSG,
3     STATE_TX_NODE,
4     STATE_RX_NODE,
5     STATE_WAIT_CONFIRM
6 } RxState;
7
8 void receive() {
9     uint8_t dataArr[2] = {0};
10    uint8_t confirmBuf[1] = {0};
11    RxState state = STATE_WAIT_MSG;
12    uint32_t timeout_ms = 300;
13
14    radio.startReceive();
15
16    while (true) {
17        switch (state) {
18            case STATE_WAIT_MSG:
19                if (irqFired) {
20                    radio.readData(dataArr, 2);
21                    state = STATE_TX_NODE;
22                }
23                break;
24
25            case STATE_TX_NODE:
26                radio.startTransmit(dataArr, 2);
27                waitForIRQ(RADIOLIB_IRQ_TX_DONE);
28                radio.startReceive();
29                state = STATE_WAIT_CONFIRM;
```

```
30     timeout_start = millis();
31     break;
32
33     case STATE_WAIT_CONFIRM:
34         if (millis() - timeout_start > timeout_ms) {
35             memset(dataArr, 0, sizeof(dataArr));
36             radio.startReceive();
37             state = STATE_WAIT_MSG;
38             break;
39         }
40         if (irqFired) {
41             radio.readData(confirmBuf, 1);
42             if (confirmBuf[0] == 0xAA) {
43                 Serial.printf("CMD=0x%02X VAL=0x%02X\n",
44                             dataArr[0], dataArr[1]);
45             }
46             memset(dataArr, 0, sizeof(dataArr));
47             radio.startReceive();
48             state = STATE_WAIT_MSG;
49         }
50         break;
51     }
52 }
53 }
```

Listing C.1 – Machine d'états RX (structure simplifiée)

Annexe D

Mesures RSSI – Balayage complet des paramètres LoRa

Table D.1 – Récapitulatif des mesures RSSI pour les 128 configurations LoRa (50 paquets par configuration, distance fixe de 3 m en espace libre)

ID	SF	BW (kHz)	CR	RSSI moyen (dBm)	Écart-type (dB)
1	5	203	4/5	-45.84	0.68
2	5	203	4/6	-45.60	1.16
3	5	203	4/7	-45.06	1.08
4	5	203	4/8	-47.14	0.99
5	5	406	4/5	-45.18	0.75
6	5	406	4/6	-44.84	0.77
7	5	406	4/7	-45.32	1.27
8	5	406	4/8	-47.54	1.51
9	5	812	4/5	-45.58	1.21
10	5	812	4/6	-44.68	0.79
11	5	812	4/7	-45.06	0.55
12	5	812	4/8	-46.34	0.89
13	5	1625	4/5	-46.06	0.24
14	5	1625	4/6	-46.12	0.33
15	5	1625	4/7	-46.50	0.51
16	5	1625	4/8	-46.74	0.44
17	6	203	4/5	-45.66	0.52
18	6	203	4/6	-45.80	1.05
19	6	203	4/7	-46.40	1.05
20	6	203	4/8	-46.94	0.31

Suite page suivante

Table D.1 – suite

ID	SF	BW (kHz)	CR	RSSI moyen (dBm)	Écart-type (dB)
21	6	406	4/5	-45.92	0.54
22	6	406	4/6	-45.98	0.14
23	6	406	4/7	-45.84	0.37
24	6	406	4/8	-45.42	0.50
25	6	812	4/5	-45.88	0.33
26	6	812	4/6	-45.76	0.43
27	6	812	4/7	-45.36	0.56
28	6	812	4/8	-45.78	0.42
29	6	1625	4/5	-46.00	0.00
30	6	1625	4/6	-46.00	0.00
31	6	1625	4/7	-45.86	0.50
32	6	1625	4/8	-45.18	0.39
33	7	203	4/5	-44.66	0.48
34	7	203	4/6	-44.92	0.27
35	7	203	4/7	-44.76	0.43
36	7	203	4/8	-44.92	0.27
37	7	406	4/5	-45.76	0.80
38	7	406	4/6	-46.16	0.62
39	7	406	4/7	-45.54	0.50
40	7	406	4/8	-46.28	0.61
41	7	812	4/5	-46.40	0.99
42	7	812	4/6	-46.04	0.45
43	7	812	4/7	-46.12	1.04
44	7	812	4/8	-45.94	0.37
45	7	1625	4/5	-46.00	0.00
46	7	1625	4/6	-46.96	0.81
47	7	1625	4/7	-46.90	0.61
48	7	1625	4/8	-46.10	0.71
49	8	203	4/5	-46.84	0.47
50	8	203	4/6	-47.06	0.24
51	8	203	4/7	-47.02	0.14
52	8	203	4/8	-47.78	0.42
53	8	406	4/5	-47.96	0.49
54	8	406	4/6	-47.14	0.61
55	8	406	4/7	-46.90	1.36

Suite page suivante

Table D.1 – suite

ID	SF	BW (kHz)	CR	RSSI moyen (dBm)	Écart-type (dB)
56	8	406	4/8	-47.20	0.45
57	8	812	4/5	-47.24	0.43
58	8	812	4/6	-46.96	0.60
59	8	812	4/7	-47.36	0.92
60	8	812	4/8	-47.72	0.54
61	8	1625	4/5	-48.82	0.39
62	8	1625	4/6	-48.64	0.48
63	8	1625	4/7	-48.46	0.61
64	8	1625	4/8	-49.02	0.25
65	9	203	4/5	-47.68	0.55
66	9	203	4/6	-47.54	1.31
67	9	203	4/7	-46.98	1.19
68	9	203	4/8	-44.92	0.90
69	9	406	4/5	-44.04	0.53
70	9	406	4/6	-45.50	1.25
71	9	406	4/7	-45.50	0.51
72	9	406	4/8	-45.82	0.39
73	9	812	4/5	-46.86	0.67
74	9	812	4/6	-46.06	0.24
75	9	812	4/7	-46.16	0.37
76	9	812	4/8	-46.06	0.24
77	9	1625	4/5	-46.02	0.14
78	9	1625	4/6	-46.82	0.69
79	9	1625	4/7	-46.02	0.14
80	9	1625	4/8	-46.04	0.20
81	10	203	4/5	-46.86	0.40
82	10	203	4/6	-46.46	0.50
83	10	203	4/7	-46.72	0.54
84	10	203	4/8	-47.06	0.31
85	10	406	4/5	-46.38	0.49
86	10	406	4/6	-46.62	0.49
87	10	406	4/7	-46.44	0.50
88	10	406	4/8	-46.52	0.50
89	10	812	4/5	-46.74	0.44
90	10	812	4/6	-47.16	0.65

Suite page suivante

Table D.1 – suite

ID	SF	BW (kHz)	CR	RSSI moyen (dBm)	Écart-type (dB)
91	10	812	4/7	-46.92	0.27
92	10	812	4/8	-47.04	0.40
93	10	1625	4/5	-46.90	0.30
94	10	1625	4/6	-46.30	0.46
95	10	1625	4/7	-46.74	0.44
96	10	1625	4/8	-46.60	0.83
97	11	203	4/5	-46.18	0.44
98	11	203	4/6	-46.50	0.81
99	11	203	4/7	-45.56	0.50
100	11	203	4/8	-45.66	0.52
101	11	406	4/5	-45.52	0.61
102	11	406	4/6	-45.92	0.44
103	11	406	4/7	-46.04	0.28
104	11	406	4/8	-45.24	0.43
105	11	812	4/5	-45.62	0.49
106	11	812	4/6	-45.44	0.50
107	11	812	4/7	-45.76	0.43
108	11	812	4/8	-45.78	0.46
109	11	1625	4/5	-45.50	0.51
110	11	1625	4/6	-45.96	0.20
111	11	1625	4/7	-45.98	0.14
112	11	1625	4/8	-45.90	0.30
113	12	203	4/5	-45.38	0.60
114	12	203	4/6	-45.72	0.54
115	12	203	4/7	-45.90	0.51
116	12	203	4/8	-46.20	0.45
117	12	406	4/5	-45.94	0.24
118	12	406	4/6	-45.24	0.94
119	12	406	4/7	-45.32	0.79
120	12	406	4/8	-44.66	0.87
121	12	812	4/5	-44.42	0.67
122	12	812	4/6	-46.24	1.08
123	12	812	4/7	-47.42	0.67
124	12	812	4/8	-46.68	0.89
125	12	1625	4/5	-45.46	0.84

Suite page suivante

Table D.1 – suite

ID	SF	BW (kHz)	CR	RSSI moyen (dBm)	Écart-type (dB)
126	12	1625	4/6	-46.00	0.20
127	12	1625	4/7	-46.64	0.48
128	12	1625	4/8	-46.04	0.20